

CONTENTS

Semana 2 — Análisis de Algoritmos y Orden de Crecimiento	
Programación 3 — FING UdelaR · Guía de la semana	
Checklist — qué hay que saber	
Teoría condensada (notación del curso)	
Ejercicios del Práctico 2 — tabla de dificultad	
Dónde aparece este tema en parciales y exámenes anteriores	

Semana 2 — Análisis de Algoritmos y Orden de Crecimiento

PROGRAMACIÓN 3 — FING UDELAR · GUÍA DE LA SEMANA

Checklist — qué hay que saber

- ☐ Definiciones formales de O , Ω , Θ , o (oh-chica) — con cuantificadores explícitos ($\forall/\exists$, orden correcto).
- ☐ Diferencia clave: O necesita **una** constante c que funcione; o necesita que la cota valga para **cualquier** $\varepsilon > 0$ (cota “no ajustada”, crecimiento estrictamente más lento).
- ☐ Relación $o(f) \Rightarrow O(f)$, pero $O(f) \not\Rightarrow o(f)$ en general.
- ☐ Propiedades algebraicas de $O/\Omega/\Theta$ bajo composición: cuadrado, potencia, exponencial, logaritmo — **no todas se preservan igual**.
- ☐ Saber construir el caso peor (worst-case input) de un algoritmo para probar una cota Ω ajustada — no alcanza con acotar arriba, hay que exhibir una entrada que fuerce ese costo.
- ☐ Argumento de independencia de la implementación: si la cota Ω surge de una propiedad intrínseca del algoritmo (cantidad de pasos que *cualquier* implementación correcta debe ejecutar), la cota vale para toda implementación.

Teoría condensada (notación del curso)

$T(n)$ es $O(f(n))$ si $\exists c > 0, n_0 \geq 0$ tal que $\forall n \geq n_0: T(n) \leq c \cdot f(n)$. **$T(n)$ es $\Omega(f(n))$** si $\exists c > 0, n_0 \geq 0$ tal que $\forall n \geq n_0: T(n) \geq c \cdot f(n)$. **$T(n)$ es $\Theta(f(n))$** si es $O(f(n))$ y $\Omega(f(n))$ simultáneamente. **$T(n)$ es $o(f(n))$** si $\forall \varepsilon > 0 \exists n_\varepsilon \geq 0$ tal que $\forall n \geq n_\varepsilon: T(n) < \varepsilon \cdot f(n)$.

Por qué o es más fuerte que O : en O alcanza con hallar una constante c que funcione (podés elegirla vos, una sola vez). En o , la desigualdad tiene que valer para **cualquier** ε , por más chico que sea — eso obliga a que el cociente $T(n)/f(n)$ tienda a 0, no solo que quede acotado.

Patrón para probar una cota Ω ajustada de un algoritmo: no alcanza con decir “en el peor caso puede tardar esto” de forma vaga — hay que (1) construir una familia de instancias explícita parametrizada por n , (2) mostrar que sobre esa familia el algoritmo ejecuta al menos

$k \cdot n^d$ pasos, con k, n_0 constantes. El ejemplo clásico de este práctico: listas de preferencia de Gale-Shapley donde cada hombre debe agotar casi toda su lista, forzando $\sum_i (n-i+1) = \Theta(n^2)$ propuestas totales.

Ejercicios del Práctico 2 — tabla de dificultad

Ejercicio	Tema	Dificultad	Notas
1 (KT Ex. 2.5)	V/F sobre preservación de O bajo cuadrado, exponencial, logaritmo	★★	4 afirmaciones — la (c) y (d) son casi iguales pero una tiene una hipótesis extra que cambia todo
2	Cota $\Omega(n^2)$ para Gale-Shapley + independencia de implementación	★★	Requiere construir el caso peor explícito (no alcanza con argumentar en general)
3	Demostraciones formales sobre o-chica (6 partes)	★★	Manipulación directa de cuantificadores; (c) y (f) piden contraejemplos/ejemplos concretos

(Resolución completa y verificada paso a paso: ver [Practico2_Resolucion.pdf](#) en [Guías de Estudio Programación 3](#).)

Dónde aparece este tema en parciales y exámenes anteriores

El análisis de complejidad y la notación asintótica casi nunca son el “problema grande” de un examen — son la **herramienta transversal** que se usa dentro de la demostración de complejidad de *cualquier otro* ejercicio (grafos, greedy, DP, flujo). Por eso, dominar bien las definiciones formales (poder escribir el cuantificador correcto sin dudar) rinde en **todos** los ejercicios del curso, no solo en éste. Vale la pena automatizar el patrón “asumo $T(n)$ es $X(f(n))$, aplico la definición con sus cuantificadores, concluyo $Y(g(n))$ ” — es el que se repite en cada demostración de complejidad de aquí en adelante.